

AIT 500 Course Project

AI-Powered Multi-Condition Health & Food Safety App Student Project Assignment Briefs

Project Overview

VitalScan is a health companion app that uses face scanning and food scanning to help people with chronic conditions like diabetes and hypertension make safer food choices in real time. A 30-second face scan reads your biomarkers, then scanning a food barcode or photo tells you how that food impacts your specific conditions right now. This project is divided into four parallel AI workstreams, each owned by a different student group.

Group 1 rPPG Signal Extraction	Group 2 Food Recognition AI	Group 3 Biomarker Risk Prediction	Group 4 NLP + Prompt Engineering
--	---------------------------------------	---	--

Shared API Contract

All groups work against this shared JSON contract from day 1. Groups 3 and 4 (instructor-owned) consume the outputs of Groups 1 and 2. Until real outputs are ready, groups work against mock data matching this schema.

```
{ "biomarkers": { "heart_rate": 72, "hrv_sdn": 45, "stress_index": 0.6,
"blood_pressure": { "systolic": 138, "diastolic": 88 } }, "food": { "name": "Canned
Chicken Soup", "sodium_mg": 890, "sugar_g": 4, "carbs_g": 12, "calories": 180 },
"conditions": ["diabetes", "hypertension"] }
```

rPPG Signal Extraction

Extract vital signs from facial video using AI signal processing

Assignment Summary

Build an AI pipeline that takes a 30-second facial video from a smartphone camera and outputs a JSON object containing heart rate, HRV, and stress index. This is the data source that powers the entire VitalScan app.

Background

Remote PhotoPlethysmoGraphy (rPPG) is a computer vision technique that extracts physiological signals — primarily the cardiac pulse — from subtle color changes in facial skin. When blood flows through capillaries with each heartbeat, it causes tiny variations in how the skin absorbs and reflects light. These variations, invisible to the human eye, can be detected by analyzing pixel color values across video frames over time.

Your group will implement this pipeline using classical signal processing algorithms and optionally extend it with a deep learning model. The output must conform to the shared API contract and be consumable by the instructor-owned condition impact engine.

Learning Objectives

- Understand the physics of photoplethysmography and why skin color changes with heartbeat
- Implement computer vision face detection and region of interest (ROI) extraction
- Apply signal processing techniques including bandpass filtering and Fast Fourier Transform (FFT)
- Extract heart rate, HRV, and stress index from a raw PPG waveform
- Evaluate model accuracy using Mean Absolute Error (MAE) against ground truth data
- Understand the gap between classical algorithms and deep learning approaches for biosignal extraction

Tasks

Task 1 — Face Detection and ROI Extraction (Core)

Use MediaPipe FaceMesh or dlib to detect facial landmarks from a live camera feed or video file. Extract the region of interest (ROI) — specifically the forehead and cheek regions — which produce the strongest rPPG signal. The ROI should track the face across frames and handle minor head movements.

- Install and configure MediaPipe FaceMesh

- Define and extract ROI bounding boxes for forehead and left/right cheeks
- Compute the mean RGB value across all pixels in the ROI per frame
- Output a time-series array of RGB values across the video duration

Task 2 — rPPG Heart Rate Extraction (Core)

Implement the POS (Plane Orthogonal to Skin) algorithm to extract the pulse waveform from the RGB time series. Apply a bandpass filter (0.7–4.0 Hz, corresponding to 42–240 BPM) to remove noise, then apply FFT to convert the waveform to the frequency domain and identify the dominant frequency as the heart rate.

- Implement the POS algorithm: project RGB signals onto a plane orthogonal to skin color
- Apply a Butterworth bandpass filter (0.7–4.0 Hz) to the resulting signal
- Apply FFT and identify the peak frequency in the valid heart rate band
- Convert peak frequency to BPM and output as `heart_rate` in the JSON contract

Task 3 — HRV and Stress Index Extraction (Core)

From the pulse waveform, detect individual heartbeat peaks (R-peaks equivalent) and compute inter-beat intervals (IBI). Use the IBI time series to calculate SDNN (standard deviation of normal-to-normal intervals) as the HRV metric. Derive a stress index from the ratio of low-frequency to high-frequency HRV power.

- Apply peak detection on the filtered PPG waveform to identify individual beats
- Compute inter-beat intervals (IBI) in milliseconds between consecutive peaks
- Calculate SDNN from the IBI series and output as `hrv_sdnn`
- Compute LF/HF power ratio using frequency band analysis and output as `stress_index` (0.0–1.0)

Task 4 — Accuracy Evaluation (Core)

Validate your pipeline against the SCAMPS synthetic dataset (which includes ground truth PPG waveforms) and optionally the UBFC-rPPG real-subject dataset. Report Mean Absolute Error (MAE) and Root Mean Square Error (RMSE) for heart rate predictions. Compare your classical algorithm performance against a published baseline.

- Download and preprocess the SCAMPS dataset
- Run your pipeline on at least 50 SCAMPS videos and compute MAE vs ground truth heart rate
- Compare against the CHROM baseline algorithm from the rPPG-Toolbox
- Report accuracy in a table: Algorithm | MAE (BPM) | RMSE (BPM)

Task 5 — Deep Learning Upgrade (Stretch Goal)

Replace the classical POS algorithm with a pretrained deep learning model from the rPPG-Toolbox (PhysNet or TS-CAN). Fine-tune on SCAMPS training data and compare accuracy against your classical implementation. Discuss the trade-offs: accuracy vs computational cost, training data requirements, and interpretability.

- Set up rPPG-Toolbox and load a pretrained PhysNet or TS-CAN model

- Fine-tune on SCAMPS training split using the toolbox training pipeline
- Evaluate on SCAMPS test split and UBFC-rPPG and compare MAE to classical method
- Write a 1-page comparison: when would you use classical vs deep learning?

Datasets

Dataset	Link	Access
SCAMPS (2,800 synthetic videos)	https://github.com/danmcduff/scampsdata	Free — instant
rPPG-Toolbox (training framework)	https://github.com/ubicomplab/rPPG-Toolbox	Free — instant
UBFC-rPPG (42 real subjects)	https://sites.google.com/view/ybenezeth/ubfcrppg	Email request — send day 1
MCD-rPPG (600 subjects + biomarkers)	https://huggingface.co/datasets/kyegorov/mcd_rppg	Free — Hugging Face

Day 1 Action Required

Email the UBFC-rPPG dataset authors immediately on day 1 to request access. Use your institutional email and describe the academic course context. Access typically arrives within 2–5 days. Begin Task 1 and 2 using SCAMPS while waiting.

Technical Stack

- Language: Python 3.10+
- Computer vision: OpenCV, MediaPipe
- Signal processing: NumPy, SciPy (butter, filtfilt, fft)
- Deep learning (stretch): PyTorch, rPPG-Toolbox
- API output: FastAPI or Flask exposing a /scan POST endpoint

Deliverables

1. Working demo — a Python script or Jupyter notebook that takes a 30-second video file and outputs a valid biomarker JSON object matching the shared contract
2. Accuracy report — a table showing MAE and RMSE on at least 50 SCAMPS test videos, compared against at least one baseline algorithm
3. Technical writeup (2–3 pages) — explaining the rPPG signal chain: face detection → ROI extraction → signal processing → frequency analysis → biomarker output
4. REST API endpoint — /scan POST that accepts a video file and returns the biomarker JSON
5. Stretch: deep learning comparison report showing accuracy improvement over classical method

Grading Rubric

Component	Weight	What we look for
Face detection + ROI extraction	20%	Face tracked correctly across frames, ROI covers forehead/cheeks, handles minor motion
Heart rate accuracy (MAE)	25%	MAE < 10 BPM on SCAMPS test set earns full marks. MAE < 5 BPM earns stretch credit
HRV + stress index output	20%	SDNN and stress index computed correctly from IBI series and output in contract format
Accuracy evaluation	15%	MAE/RMSE reported on 50+ videos with comparison to baseline algorithm
REST API + JSON contract	10%	Endpoint accepts video, returns valid JSON matching shared contract schema
Technical writeup	10%	Clear explanation of each stage in the pipeline with diagrams
Stretch: deep learning	+10%	Accuracy comparison of classical vs DL model with discussion of trade-offs

Food Recognition AI

Classify food and extract nutrition data from photos and barcodes

Assignment Summary

Build an AI pipeline that accepts either a barcode scan or a food photo and returns a structured nutrition JSON object. This data feeds directly into the instructor-owned condition impact engine, which scores the food against the user's health conditions.

Background

Accurate food recognition is one of the most commercially impactful applications of computer vision. Dietary tracking apps, hospital nutrition systems, and consumer health tools all rely on some form of food identification. The challenge is that food is highly variable: the same dish can look completely different depending on preparation, plating, lighting, and portion size.

Your group will train a convolutional neural network (CNN) on the Food-101 benchmark dataset, connect it to a nutrition lookup database, and expose the result as an API. You will also handle the simpler but equally important barcode-to-nutrition path using the Open Food Facts API.

Learning Objectives

- Understand transfer learning and how pretrained CNN features generalize to new classification tasks
- Fine-tune MobileNetV3 or EfficientNet on a real food classification benchmark
- Design a confidence-thresholding strategy to handle uncertain predictions gracefully
- Integrate a machine learning model with an external nutrition database API
- Evaluate classification accuracy using top-1 and top-5 accuracy metrics
- Understand dataset bias and its implications for real-world deployment

Tasks

Task 1 — Food Image Classifier (Core)

Fine-tune a pretrained MobileNetV3-Large or EfficientNet-B0 model on the Food-101 dataset. Food-101 contains 101,000 images across 101 food categories. Use the Hugging Face datasets library to load Food-101 directly. Freeze the backbone initially, train the classification head, then unfreeze and fine-tune end-to-end.

- Load Food-101 via: `from datasets import load_dataset; ds = load_dataset('ethz/food101')`
- Load pretrained MobileNetV3: `torchvision.models.mobilenet_v3_large(pretrained=True)`
- Replace the final classification layer with a new `Linear(num_features, 101)` layer
- Train in two phases: (1) head only for 5 epochs, (2) full fine-tune for 10 epochs
- Target: top-1 accuracy > 75%, top-5 accuracy > 92% on Food-101 test split

Task 2 — Confidence Thresholding and Fallback (Core)

A food classifier should not return a low-confidence prediction with no warning. Design a thresholding strategy: if the top-1 softmax probability is below 0.6, return the top-3 candidates with their confidence scores and ask the user to confirm. If below 0.3, flag as unrecognized and fall back to a text search.

- Implement softmax confidence extraction from model output logits
- Define three confidence tiers: High (> 0.6), Medium ($0.3\text{--}0.6$), Low (< 0.3)
- For Medium confidence: return top-3 labels with scores for user confirmation
- For Low confidence: return 'food_unrecognized' flag in the JSON output

Task 3 — Nutrition Lookup and Data Normalization (Core)

Map the classified food label to a nutrition entry. Use the USDA FoodData Central as the primary source and Open Food Facts API for packaged foods (barcode path). Normalize all outputs to a standard per-serving schema. Handle missing values gracefully with null rather than fabricated estimates.

- Download USDA FoodData Central CSV and build a local label-to-nutrition lookup table
- For the barcode path: call Open Food Facts API with the barcode and extract nutrition data
- Normalize output to the shared contract schema: sodium_mg, sugar_g, carbs_g, calories, fat_g per serving
- If a nutrient value is missing, return null rather than 0 or an estimate

Task 4 — Model Accuracy Evaluation (Core)

Evaluate your fine-tuned model on the Food-101 test split (25,250 images across 101 categories). Report top-1 and top-5 accuracy. Identify the 10 categories your model performs worst on and analyze why — is it visual similarity, label noise, or underrepresentation in training data?

- Run inference on the full Food-101 test set and compute top-1 and top-5 accuracy
- Generate a confusion matrix and identify the 10 most commonly confused food pairs
- Analyze and report: which categories have accuracy below 60% and what is the likely cause?
- Compare your results to the published Food-101 benchmark leaderboard on Papers With Code

Task 5 — Multimodal Food Understanding (Stretch Goal)

Some foods — mixed salads, stews, homemade dishes — cannot be reliably classified into a single category. Use a vision-language model (CLIP or LLaVA) to describe the food in natural language and extract nutrition estimates from the description. Compare accuracy against the CNN classifier on a set of 50 complex meal photos.

- Load OpenAI CLIP: pip install transformers and use CLIPModel + CLIPProcessor
- Prompt the model: 'Describe the food in this image and estimate its main ingredients'
- Parse the natural language output to extract likely food categories and portion cues

- Compare against CNN classifier on 50 manually-labelled complex meal photos

Datasets

Dataset	Link	Access
Food-101 (Hugging Face)	https://huggingface.co/datasets/ethz/food101	Free — one line of Python
USDA FoodData Central	https://fdc.nal.usda.gov/download-datasets/	Free — CSV/JSON download
Open Food Facts API	https://world.openfoodfacts.org/data	Free — no API key needed

Dataset Bias — Important Note for Your Report

Food-101 images were collected from a North American and European social food photography platform. Your model will likely underperform on dishes from Asian, African, Middle Eastern, and Latin American cuisines that are underrepresented in training data. You must acknowledge this limitation in your writeup and discuss how it would affect real-world deployment.

Technical Stack

- Language: Python 3.10+
- Deep learning: PyTorch + torchvision, or TensorFlow/Keras
- Dataset loading: Hugging Face datasets library
- Pretrained models: torchvision.models (MobileNetV3, EfficientNet)
- Multimodal stretch: HuggingFace transformers (CLIP)
- API output: FastAPI or Flask exposing /scan/barcode and /scan/photo endpoints

Deliverables

6. Trained model — fine-tuned MobileNetV3 or EfficientNet saved as a .pt checkpoint, achieving > 75% top-1 accuracy on the Food-101 test split
7. Accuracy report — top-1 and top-5 accuracy, confusion matrix, and analysis of worst-performing categories
8. REST API — two endpoints: POST /scan/photo (accepts image file) and POST /scan/barcode (accepts barcode string), both returning the shared contract nutrition JSON
9. Technical writeup (2–3 pages) — explaining transfer learning, confidence thresholding strategy, and dataset bias analysis
10. Stretch: multimodal comparison showing CLIP vs CNN accuracy on complex meals

Grading Rubric

Component	Weight	What we look for
-----------	--------	------------------

Model training + fine-tuning	25%	Correct two-phase training, model converges, top-1 accuracy > 75% on test split
Confidence thresholding	15%	Three-tier strategy implemented, graceful handling of uncertain and unrecognized foods
Nutrition lookup + normalization	20%	USDA + Open Food Facts integrated, output matches shared contract schema, nulls used for missing values
Accuracy evaluation	20%	Full test set evaluation, confusion matrix, worst-category analysis with explanations
REST API endpoints	10%	Both /scan/photo and /scan/barcode endpoints functional and returning valid JSON
Technical writeup	10%	Transfer learning explained, bias limitation acknowledged and discussed
Stretch: multimodal (CLIP)	+10%	CLIP implemented and compared to CNN on complex meals with qualitative analysis

Biomarker Risk Prediction

Predict condition risk from vital signs using supervised machine learning

Assignment Summary

Build a machine learning model that takes a set of biomarkers (heart rate, HRV, blood pressure, stress index) and predicts the risk level for diabetes and hypertension. This model sits between the face scan output and the instructor-owned recommendation engine, providing a risk score for each condition.

Background

Predicting chronic disease risk from physiological measurements is one of the most established applications of machine learning in healthcare. Tabular health datasets — with features like blood pressure, cholesterol, BMI, and glucose — have been used to train classifiers for over 30 years. The challenge is not just accuracy but explainability: in a health context, a model that says 'you are at risk' must also explain why.

Your group will train and evaluate multiple ML classifiers on two public health datasets, use SHAP to explain model predictions, and expose the result as a real-time risk scoring API. The output feeds directly into the instructor's condition impact engine.

Learning Objectives

- Understand supervised classification for tabular health data
- Train, tune, and compare Random Forest, XGBoost, and optionally a neural network
- Apply SHAP (SHapley Additive exPlanations) to explain model decisions
- Evaluate classifiers using ROC-AUC, precision, recall, and F1 score
- Understand the clinical significance of false negatives vs false positives in health AI
- Serve a trained ML model as a real-time REST API endpoint

Tasks

Task 1 — Data Loading and Preprocessing (Core)

Load both the UCI Heart Disease dataset and the Pima Indians Diabetes dataset. Explore each dataset, handle missing values, normalize continuous features, and encode categorical variables. Perform exploratory data analysis (EDA) to understand the distribution of each feature and its correlation with the target label.

- Load UCI Heart Disease: `from ucimlrepo import fetch_ucirepo; heart = fetch_ucirepo(id=45)`
- Load Pima Diabetes from Kaggle CSV: `pd.read_csv('diabetes.csv')`
- Handle missing values: MICE imputation or median fill — document your choice and justify it

- Normalize continuous features (age, BP, cholesterol) using StandardScaler
- Plot feature distributions and a correlation heatmap for each dataset

Task 2 — Train Multiple Classifiers (Core)

Train three classifiers on each dataset: Logistic Regression (baseline), Random Forest, and XGBoost. Use 5-fold cross-validation to report performance. Tune hyperparameters using GridSearchCV or RandomizedSearchCV for the Random Forest and XGBoost models. Report ROC-AUC, precision, recall, and F1 for each.

- Train Logistic Regression as a baseline and report all metrics
- Train Random Forest with hyperparameter search over `n_estimators`, `max_depth`, `min_samples_split`
- Train XGBoost with search over `learning_rate`, `max_depth`, `n_estimators`, `subsample`
- Use 5-fold stratified cross-validation for all models
- Report a comparison table: Model | ROC-AUC | Precision | Recall | F1

Task 3 — Explainable AI with SHAP (Core)

Apply SHAP to your best-performing model for each dataset. Generate a summary plot showing which features most influence the model's predictions. Generate a waterfall plot for at least 3 individual predictions, showing exactly how each feature contributed to that specific risk score. Discuss the clinical meaning of the top features.

- Install SHAP: `pip install shap`
- Compute SHAP values: `explainer = shap.TreeExplainer(model); shap_values = explainer(X_test)`
- Generate and save `shap.summary_plot` for both datasets
- Generate `shap.waterfall_plot` for 3 high-risk and 3 low-risk predictions
- Write a paragraph explaining each dataset's top 3 most important features in clinical terms

Task 4 — Real-Time Risk Scoring API (Core)

Serialize your best model using joblib or pickle and expose it via a REST API. The endpoint should accept a biomarker JSON object (matching the shared contract output from Group 1) and return a risk score between 0 and 1 for each condition. Include confidence intervals where possible.

- Serialize best model: `joblib.dump(model, 'risk_model.pkl')`
- Build a FastAPI endpoint: `POST /predict/risk` accepting the shared biomarker JSON
- Map Group 1's biomarker fields (`heart_rate`, `hrv_sdn`, `stress_index`) to model features
- Return: `{ 'diabetes_risk': 0.72, 'hypertension_risk': 0.85, 'confidence': 'high' }`

Task 5 — Neural Network Comparison (Stretch Goal)

Build a small fully-connected neural network (2–3 hidden layers) in PyTorch or Keras and compare its performance against your best tree-based model. Use the same train/test splits and evaluation metrics. Discuss when a neural network is preferable to gradient boosting on tabular health data and when it is not.

- Build a 3-layer MLP: input → 64 → 32 → output (sigmoid for binary classification)
- Train with Adam optimizer, binary cross-entropy loss, early stopping on validation loss
- Compare ROC-AUC against Random Forest and XGBoost on the same test split
- Write a half-page discussion: tabular data and the 'gradient boosting wins' phenomenon

Datasets

Dataset	Link	Access
UCI Heart Disease	https://archive.ics.uci.edu/dataset/45/heart+disease	Free — CC BY 4.0
Pima Indians Diabetes (Kaggle)	https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database	Free — Kaggle account needed
UCI Diabetes (backup)	https://archive.ics.uci.edu/dataset/34/diabetes	Free — CC BY 4.0

Clinical Note — False Negatives Matter More

In a health app, a false negative (telling a high-risk user they are low-risk) is far more dangerous than a false positive. When optimizing your model, prioritize recall over precision for the positive (at-risk) class. You should tune your classification threshold to achieve recall > 0.80 even if it slightly reduces overall accuracy. Document and justify this decision in your writeup.

Technical Stack

- Language: Python 3.10+
- ML: scikit-learn (LogisticRegression, RandomForestClassifier), xgboost
- Explainability: shap
- Data: pandas, numpy, matplotlib, seaborn
- Stretch neural network: PyTorch or TensorFlow/Keras
- API: FastAPI + joblib for model serving

Deliverables

11. Trained models — serialized .pkl files for the best model on each dataset (heart disease and diabetes)
12. Model comparison report — table of ROC-AUC, precision, recall, F1 across all three classifiers, with justification for final model selection
13. SHAP analysis — summary plots and waterfall plots for both datasets, with written clinical interpretation of top features
14. REST API endpoint — POST /predict/risk accepting biomarker JSON and returning condition risk scores
15. Technical writeup (2–3 pages) — covering data preprocessing decisions, model selection rationale, clinical implications of false negatives, and explainability findings
16. Stretch: neural network vs tree-based comparison with discussion

Grading Rubric

Component	Weight	What we look for
Data preprocessing + EDA	15%	Missing values handled correctly, features normalized, EDA plots showing distributions and correlations
Model training + tuning	25%	Three classifiers trained, hyperparameter search performed, cross-validation used correctly
ROC-AUC performance	20%	Best model achieves ROC-AUC > 0.80 on both datasets. > 0.85 earns stretch credit
SHAP explainability	20%	Summary and waterfall plots generated, top features interpreted in clinical terms
REST API endpoint	10%	Endpoint accepts biomarker JSON, returns risk scores in correct format
Technical writeup	10%	False negative rationale included, model selection justified, preprocessing documented
Stretch: neural network	+10%	MLP trained and compared to best tree model, tabular ML discussion included

NLP + Prompt Engineering

Generate personalized food safety explanations using large language models

Assignment Summary

Design, test, and evaluate a prompt engineering system that uses a large language model (LLM) to generate clear, accurate, personalized explanations of why a specific food is risky for a specific user given their conditions and current vitals. This is the user-facing layer of the VitalScan app — what the user actually reads.

Background

Large language models have transformed how AI systems communicate with users. In healthcare, the challenge is not just generating fluent text — it is generating text that is accurate, grounded in medical evidence, appropriately cautious, and personalized to the user's specific situation. A system that tells a hypertensive user 'this soup is salty' is far less useful than one that explains 'this soup contains 890mg of sodium — 59% of your daily limit — and your blood pressure scan showed elevated readings this morning, making this particularly risky today.'

Your group will design and systematically evaluate prompts for this task, implement retrieval-augmented generation (RAG) to ground outputs in verified medical guidelines, and build a hallucination detection test to measure factual accuracy. This is not a coding-heavy project — it is a thinking-heavy project about how to make AI systems trustworthy.

Learning Objectives

- Understand prompt engineering as a systematic discipline, not trial and error
- Design structured output prompts using function calling / JSON mode
- Evaluate LLM outputs systematically across multiple dimensions: accuracy, personalization, tone, safety
- Implement retrieval-augmented generation (RAG) using vector embeddings and a document store
- Build a hallucination detection test and measure factual accuracy across prompt strategies
- Understand the ethical responsibilities of deploying LLMs in health contexts

Tasks

Task 1 — Prompt Design and Systematic Evaluation (Core)

Design at least 4 distinct prompt strategies for generating food safety explanations. Each strategy should produce a different type of output. Test each strategy on a manually-created evaluation set of 50 food-condition-vitals combinations. Score each output on accuracy, personalization, clarity, and safety using a rubric you define.

- Define 4 prompt strategies: (1) zero-shot, (2) few-shot with examples, (3) chain-of-thought, (4) structured role-play as a dietitian
- Build an eval set of 50 test cases: each case specifies food nutrition, user conditions, and current vitals
- Define a scoring rubric: Accuracy (0–3), Personalization (0–3), Clarity (0–2), Safety (0–2) = 10 points max
- Score all 4 strategies on all 50 cases. Report mean score per strategy and per dimension
- Identify which strategy performs best overall and in which dimensions each strategy excels

Task 2 — Structured Output Generation (Core)

A free-text explanation is hard to integrate into an app. Design a prompt that causes the LLM to return structured JSON with separate explanation fields for each condition, a risk level (low/medium/high), and a suggested alternative food. Use function calling (Anthropic tool use or OpenAI function calling) to enforce the schema.

- Design a JSON output schema: { risk_level, diabetes_explanation, hypertension_explanation, alternative_food, confidence }
- Implement using Anthropic tool use or OpenAI function calling to enforce the schema
- Test on 20 cases and verify that all 20 return valid JSON matching the schema
- Handle edge cases: what happens when the LLM is uncertain? Design a graceful degradation

Task 3 — RAG with Medical Guidelines (Core)

Build a retrieval-augmented generation pipeline that grounds LLM outputs in verified dietary guidelines. Chunk the ADA nutrition guidelines and USDA Dietary Guidelines into passages, embed them using a sentence transformer, store in a vector database (ChromaDB or FAISS), and retrieve the top-3 most relevant passages for each query before generating the explanation.

- Download and chunk the ADA guidelines and USDA Dietary Guidelines 2020–2025 into 200-word passages
- Embed chunks using: from sentence_transformers import SentenceTransformer; model = SentenceTransformer('all-MiniLM-L6-v2')
- Store embeddings in ChromaDB: pip install chromadb
- At query time: embed the food-condition query, retrieve top-3 passages, inject into the prompt as context
- Compare RAG vs non-RAG outputs on 20 test cases: does RAG improve accuracy?

Task 4 — Hallucination Detection and Factual Accuracy Testing (Core)

LLMs can make confident but false claims about nutrition and medical facts. Build a test to detect hallucinations. For 30 test cases where you know the correct nutritional and medical facts, check whether the LLM output contradicts the ground truth. Report a hallucination rate for each prompt strategy and discuss how RAG affects it.

- Create 30 test cases with known correct facts: exact sodium content, correct dietary guideline limits, accurate clinical thresholds

- Run each prompt strategy and manually check each output for factual errors against the known facts
- Compute hallucination rate: number of outputs with at least one factual error / total outputs
- Compare hallucination rate: zero-shot vs RAG-augmented prompts
- Categorize types of hallucination: wrong numbers, wrong guideline thresholds, invented medical claims

Task 5 — Safety and Tone Evaluation (Stretch Goal)

A health AI must not cause harm. Design a safety test for your system: does it ever recommend unsafe foods to a high-risk user? Does it use language that could cause anxiety or discourage users from eating? Does it make medical claims it should not (e.g., 'this will cause a heart attack')? Build a systematic safety evaluation with 20 adversarial test cases.

- Design 20 adversarial test cases targeting safety: very high-risk foods, edge-case conditions, extreme vitals
- Define safety criteria: never recommends a high-sodium food to a hypertensive user with elevated BP, never makes diagnostic claims, never uses alarmist language
- Evaluate each strategy against all 20 safety cases
- Propose and implement a safety guardrail prompt that prevents the most common failure modes

Resources

Dataset	Link	Access
ADA Nutrition Guidelines	https://diabetes.org/food-nutrition	Free — PDF download
USDA Dietary Guidelines 2020–2025	https://www.dietaryguidelines.gov/resources/2020-2025-dietary-guidelines-online-materials	Free — PDF + structured data
AHA Healthy Eating	https://www.heart.org/en/healthy-living/healthy-eating	Free — reference
Anthropic Claude API	https://console.anthropic.com	Free trial credits available

Build Your Own Eval Set

There is no existing benchmark dataset for this exact task — food safety explanation for chronic conditions. Building your own 50-case evaluation set is not a limitation; it IS a core deliverable. Designing a good evaluation is a critical AI engineering skill. Your eval set should cover: multiple condition combinations, a range of food risk levels (low, medium, high), and edge cases like foods that are good for one condition but bad for another.

Technical Stack

- Language: Python 3.10+
- LLM APIs: Anthropic Claude API or OpenAI API (both have free trial credits)

- Embeddings: sentence-transformers (all-MiniLM-L6-v2)
- Vector store: ChromaDB or FAISS
- PDF parsing: PyMuPDF (fitz) or pdfplumber for chunking guidelines
- Evaluation: pandas for results tables, matplotlib for score visualizations

Deliverables

17. Prompt strategy report — 4 prompt strategies documented with examples, scored on 50 eval cases, with mean scores per strategy and per dimension in a comparison table
18. Structured output system — function-calling implementation that returns valid JSON for 20+ test cases, with schema documented
19. RAG pipeline — working ChromaDB vector store loaded with ADA and USDA guideline chunks, with retrieval-augmented generation compared to baseline
20. Hallucination test results — 30-case factual accuracy test with hallucination rate per strategy, error categorization, and RAG comparison
21. Technical writeup (2–3 pages) — covering prompt design rationale, RAG architecture, hallucination findings, and ethical considerations for health LLMs
22. Stretch: safety evaluation with 20 adversarial test cases and guardrail implementation

Grading Rubric

Component	Weight	What we look for
Prompt design + eval set	25%	4 distinct strategies, 50 well-designed eval cases, scoring rubric defined and applied consistently
Structured output (JSON schema)	15%	Function calling implemented correctly, valid JSON returned on 20+ test cases, edge cases handled
RAG pipeline	20%	Guidelines chunked and embedded correctly, retrieval working, RAG vs baseline comparison documented
Hallucination detection	20%	30-case test with factual verification, hallucination rates reported per strategy, error types categorized
Technical writeup	10%	Prompt strategies explained, RAG architecture described, ethical considerations discussed
Eval set quality	10%	50 cases cover diverse conditions, risk levels, and edge cases — not trivial or repetitive
Stretch: safety evaluation	+10%	20 adversarial cases tested, safety failures identified, guardrail prompt implemented and tested

Integration Guide — All Groups

All four groups develop their modules independently against mock data. Integration happens at the Week 5 milestone. This section describes how the groups connect.

Data Flow

Pipeline

Group 1 (face scan) → biomarker JSON → Group 3 (risk prediction) → risk scores → Instructor engine → Group 4 (NLP explanation). Group 2 (food scan) → nutrition JSON → Instructor engine → Group 4 (NLP explanation).

Mock Data for Development

Until Groups 1 and 2 deliver their APIs, Groups 3 and 4 use this mock data:

```
MOCK BIOMARKERS: {"heart_rate": 78, "hrv_sdn": 38, "stress_index": 0.71,
"blood_pressure": {"systolic": 142, "diastolic": 91}}
MOCK FOOD: {"name": "Instant Ramen", "sodium_mg": 1680, "sugar_g": 2, "carbs_g": 54,
"calories": 380, "fat_g": 14}
```

Integration Milestone — Week 5

- All groups submit their working API endpoint URL to the instructor
- Groups 3 and 4 swap mock data URLs for real Group 1 and 2 API calls
- Full end-to-end pipeline demo: face scan → food scan → risk score → explanation
- Each group presents a 10-minute demo of their module, then the class watches the integrated system run

Timeline

Component	Weight	What we look for
Week 1	All groups	Review assignment, agree on API contract, request UBFC-rPPG access, set up dev environment
Week 2	All groups	Begin core Task 1 and Task 2 using mock data
Week 3	All groups	Core tasks complete, begin Task 3 and Task 4
Week 4	All groups	All core tasks complete, begin stretch goals, prepare APIs
Week 5	All groups	Integration day — connect real APIs, run end-to-end demo, submit deliverables
Week 6	All groups	Final presentations: 15 min per group + 5 min Q&A